

Lab1

姓名：杨梦琪

学号：25300120072

一、实验思路

P1: signMask - return a mask with only the most significant bit set (0x80000000)

题目中要求返回 32 位 int,并只有最高位是 1，也就意味着 1 需要左移 31 位到 32 位处就可以实现题目中的要求。

P2: bitXor - $x \oplus y$ using only $\sim$ and $\&$

题目中要求用 $\&$ 和 $\sim$ 来达到异或的效果，当 $x=y$ 时取 0， $x \neq y$ 时取 1；这就意味着可以通过找出 x ， y 二进制中为共同为 1 和共同为 0 的部分来解决，对共同为 1 的二进制数和共同为 0 的二进制数按位与，若等于 1，说明 $x=y$ ；但题目中要求，相等时应取 0，则将共同为 1 的二进制数和共同为 0 的二进制数先各自取反，再取二者与即可。

P3: negativePart - return $-x$ if $x < 0$, otherwise return 0

题目中要求当 x 为负数则返回 $-x$, x 为正数则返回 0；所以首先判断正负，二进制数的符号位是第 31 位，若第 31 位为 1，则为负数；第 31 位为 0，则为正数。所以可以将符号位右移 31 位，如果为负数，则 31 位全为 1；如果是正数，则 31 位为 0。 x 按位取反加一即为其相反数，和右移得到的二进制数取与，则能得到 $-x$ 。

P4: copyByteWithin - copy byte src of x to byte dst, leaving all other bytes unchanged

题目要求把 x 中的 src 编号的 1 字节复制到 dst 编号的字节位置，所以思路应当是，先从 x 中取出 src 编号的 1 字节；接下来，需要把 dst 编号的字节位置清零，即先写出一个掩码，将掩码按照 dst 所标位置左移并取反，dst 字节位置保证清零。最后的操作就是，先将 x 和刚刚的掩码取与，将其他位置的填充好，再和左移到 dst 位置的 src 编号的 1 字节取或即可。

P5: logicalShift - shift x to the right by n bits, using a logical shift

按照题目要求，首先将 x 右移 n 位，但如果 x 为负数，就会导致左边多出来 n 个 1，所以需要构造一个掩码，并让掩码在左边 n 位均为 0，然后和右移得到的 x 取与就可以了。

P6: swapNibblePairs - swap the low and high 4 bits within each byte of x

这道题需要交换每个 byte 上的高位字节和低位字节，那就可以先单独把每个 byte 的低位字节和高位字节分别取出来。最低位的左移 4 位即可，但是最高位的需要右移，为了防止其为负数，需要和 0x0F0F0F0F 进行取与运算，然后得到的 high 和 low 再取或就可以了。

P7: secondLowestZeroBit - return a mask that marks the position of the second least significant 0 bit

首先由于常用方法都是寻找 1 而非 0，所以我们先对 x 取反，来找倒数第 2 低位的 1。我需要把最低位 1 去掉，所以需要让 $\sim x - 1$ ，（不能写减号，转化为 $\sim x + \sim 0$ 并和 $\sim x$ 取与，这样可以保证不生成新的 1，同时将原来的倒数第二个 1 转化为倒数第一个，现在的目的是仅需保留这个倒数第二个 1，则对清除掉最低位 1 的数取反+1 再取与，就能仅保留下倒数第二个 1。

P8: oddParity - return the odd parity bit of x , that is, when the number of 1s in the binary representation of x is even, then the return 1, otherwise return 0.

由于无法使用 while 循环函数，那就可以通过均分为两半的方法提高效率，32 为中将高 16 位和低 16 位进行异或处理，异或情况下，不同则为 1，相同则为 0，接下来将高 8 位和低 8 位、高 4 位低 4 位，高 2 位低 2 位及高 1 位及低 1 位进行异或，最终 bit0 处显示 0 即为偶数个，为 1 则为奇数个。

P9: rotateRightBits - rotate x to right by n bits

这个题目需要把低 n 位挪到高 n 位处，一个是需要拿到低 n 位的 bit，并利用掩码使之左移至高 n 位，然后把原来的 x 右移 n 位，高位均为 1，再取与即可。

P10: roundEvenPow2 - round nonnegative x to the nearest multiple of 2^n .

*** If x is exactly halfway between two multiples, choose the multiple whose quotient by 2^n is even.**

对于这道题目，我们需要找到距离非负数 x 最近的 2^n 的倍数，首先我们需要表示 2^n ，即 1 左移 n 位；其次需要找到 x 距离小于他的最近的 2^n 的倍数，使用 $x \& (s-1)$ 来找其余数即可；接下来需要找到 2^n 两个相邻的倍数之间的距离的一半，从而和余数进行比较，如果比余数小，说明 x 更近的是大于他的那个倍数；如果比余数大，说明 x 更近的是小于他的那个倍数，这个部分可以通过 `rem` 和 `half` 相减，并右移 31 位判断 1 或 0 来实现。但如果余数和距离的一半相等（使用异或来判断），根据题目的特殊规则，我们需要选择那个除以 2 为偶数的那个倍数

P11: midpointTowardFirst - return the exact mathematical midpoint $(x+y)/2$ without overflow. If the exact midpoint lies halfway between two integers, choose the adjacent integer that is closer to the first argument x .

这道题目需要先行计算 $x+y/2$ 的值，因为直接计算 $x+y$ 可能会导致溢出，我们就把他分成两个部分计算，`same` 部分通过与计算出 x 、 y 共同拥有 1 的位，这部分需要进位，所以与运算后需要左移 1 位，`diff` 部分通过异或计算出 x 、 y 不同的位，保留即可，那么 $x+y$ 就是 `same<<1+diff`，再除以 2 就是整体左移，得到 `same+diff>>1`。接下来就需要处理特殊情况，若正好计算出来的是 .5 怎么办：一方面要解决 x 和 y 的大小问题，还是通过 $x-y$ 并左移 31 位来判断；另一方面需要判断 $x+y$ 的和是不是奇数，这部分就用我们的 `diff` 来判断即可，因为 `same` 一定是偶数向前进位，`diff` 最低位若为 1，那么就是奇数。如果为奇数，就要通过判断 x 和 y 的大小来决定是否+1 进位。

但尝试一次后发现要考虑 x 和 y 的正负。一种情况是异号，如果需要进位，只能是 x 为正数， y 为负数；另一种情况是同号，必须满足 $x-y$ 大于 0 才需要进位+1，就和之前的判断方法类似了。

P12: `isBetweenEitherOrder` - return 1 when x lies in the inclusive interval whose endpoints are a and b . The endpoints may be given in either order.

对于这道题目，我们不清楚 a 和 b 谁大谁小，也不知道 a 、 b 、 x 的正负情况，所以要判断 x 是否落在闭区间里，等价于 $(x-a)$ 和 $(x-b)$ 符号相反，或者其中一个等于 0。参考上一题出现的溢出错误，不能直接做减法，要分同号、异号处理。

首先分别取出 x 、 a 、 b 的符号位。如果两个数同号，相减不会溢出，直接算差值看符号；如果两个数异号， x 的符号就是差值的符号，不用做减法，避开溢出。用这个方法分别算出 $x-a$ 和 $x-b$ 的准确符号。两个符号做异或，结果为 1 代表符号相反，说明 x 在两个端点之间；再加上 x 等于 a 、 x 等于 b 两种情况，异或判等不会溢出。最后把结果都压缩成 0 或 1，保证返回值是整数 1，不是全 1 掩码。

P13 : `mul5Sat` - return $x*5$, and if $x*5$ overflow, change the result to `INT_MAX(0x7fffffff)` or `INT_MIN(0x80000000)` correspondingly

对于这道题目，我们要算出 x 乘 5，溢出就做饱和处理，正数溢出返回 `INT_MAX`，负数溢出返回 `INT_MIN`。不能直接写乘法，只能用移位加法。 $x*5 = (x \ll 2) + x$ ，先算出乘积 `sum`。对比 `sum` 和 x 的符号，符号不一样代表溢出，右移 31 位拿到溢出掩码 `over`。构造 `INT_MIN` 和 `INT_MAX`，取出 x 的符号 `sign`。溢出时根据 x 正负选择饱和值 `sat`，不能用 `if`，依靠掩码 `&`、`|` 二选一。

P14: `classifyAdd3` - classify the exact mathematical sum $x+y+z$.

* Return 1 if the sum is greater than `INT_MAX`, -1 if it is less than `INT_MIN`, and 0 otherwise. You may not use a wider integer type.

这道题要求判断三个数字相加是否溢出，但是有一个问题是，两个数相加可能就导致溢出了，这是第一次溢出，加上第三个，还可能会有一次溢出，如果两次溢出一正一负，那么整体不溢出。所以我们要先判断两两相加是否溢出，再判断三个加在一起是否溢出。和 12 题步骤有些类似，利用右移的方法，我们先判断 x ， y ， z 三个数各自的正负情况，同时判断 $x+y$ 、 $x+y+z$ 的正负情况。

然后找出溢出不合理的情况：当 x 、 y 非负数，而 $x+y$ 是负数，则记录正溢出；同样当 x 、 y 为负数， $x+y$ 为非负数，则记录负溢出； $x+y+z$ 同理。

接下来就是找整体正溢出和负溢出的情况：对于正溢出，可能的情况是两次都是正溢出，也有可能是只有一次正溢出，另外一次没有负溢出；对于负溢出逻辑一致。

找到之后返回或，正溢出为 1，负溢出为 -1。

P15 : `floatScaleThreeHalves` - Return bit-level equivalent of expression $f*3/2$ for floating point argument f .

* Both the argument and result are passed as unsigned int's, but they are to be interpreted as the bit-level representation of single-precision floating point values.

* Use round-to-nearest-even. Preserve the sign of both +0 and -0.

* When argument is NaN, return argument.

针对于浮点型数据，我把他分成三部分，第一部分是符号位，第二部分是阶

码，也就是 0~255 范围内，第三部分是尾码。对于阶码部分等于 0xFF 的，*1.5 一定会溢出，进行单独处理；接下来是阶码为 0 的，则对其尾数进行先*3，后再除 2 来解决，最后结果需要判断是否可以使阶码升级为 1，分两种情况来输出；另外就是阶码不为 0 的，处理方法和第二种类似，阶码和尾码分开计算，最后再放在一起，但同时要注意处理是否溢出。

P16: floatRoundEven - round the floating-point value represented by uf to the nearest integer, with halfway cases rounded to the even integer. Return the bit-level representation of that integer as a single-precision float. If rounding produces zero, preserve the input sign; thus a negative value that rounds to zero returns -0. When uf is NaN or infinity, return uf unchanged.

和上一道题一样，针对于浮点型数据，分成三部分，第一部分是符号位，第二部分是阶码，也就是 0~255 范围内，第三部分是尾码。对于阶码部分等于 0xFF 的，本身就是 NaN 或者无穷大，不用做任何处理，直接返回原数；

接下来是阶码对应的真实指数大于等于 23 的，这时候小数点已经移到了尾数的最右边，没有小数部分，本身就是已经是整数，直接返回原数就行；然后是真实指数小于 0 的，数的绝对值小于 1，舍入结果就是 0，注意保留原来的符号，负数要返回 -0；另外就是指数在 0 到 22 之间的，这时候尾数里既有整数部分也有小数部分，把小数部分单独截出来，拿小数最高位当舍入位、剩下的低位当粘滞位，按照就近舍入到偶数的规则判断要不要向整数部分进位；进位的时候要注意尾数会不会溢出到隐藏位，溢出的话阶码要加 1；最后把符号位、新阶码、处理好的尾码拼在一起返回就行，同时要注意阶码溢出变成无穷大的情况。

P17: float_i2f - Return bit-level equivalent of expression (float) x.

*** Result is returned as unsigned int, but it is to be interpreted as the bit-level representation of a single-precision floating point values.**

这道题目和之前的题目有不同，一个是我们需要把输入的 x 转换为无符号数，剩下需要的数据是一样的，整理出其符号位，阶码，尾码，指数 e 的大小（以防弄混这道题我将 e 命名为了 shift，再写上一道题的时候 e 和 exp 弄混出错频次太多了），和要舍弃掉的低位。

这道题目的特殊情况，当 x=0 是，直接 return 0；

接下来针对小于 0 的特殊情况，利用无符号 0 (0U)，将 x 强制转换为符号数并存储在绝对值 abs_x 中。

以上特殊情况都处理完毕后，就是去判断这个数字 x 真实指数应当是多少。也就是我们要找到 x 二进制数最高位的 1 在什么位置。通过 1U<<shift，我们造出来了一个只有最高位是 1 的掩码，和 x 的绝对值取与，只有其最高位和 x 绝对值最高位都为 1 时，这个运算才等于 1，否则为 0，并带入循环中，只要他们运算等于 0，即为假，取非，然后 shift--，直到运算为 1，则终止循环。

接下来根据得到的指数对绝对值进行移位，我们只需要保留 23 位尾数，所以我们先需要把绝对值最高位移到最顶端，并用 frac 存储其向右移 8 位的数字，再用 round_part 存储要被舍弃的部分。

对于要被舍弃的部分，题目中有特殊要求，这部分如果大于 0.5 则 frac 进位，小于 0.5 则不进位，如果等于 0.5 且 frac 为奇数则进位。

最后把符号位，阶码和尾码拼在一起即可。

P18: bitCount - return count of number of 1's in the binary representation of x

在这道题中，我们要统计出来 x 二进制表达中 1 的个数。因为不能用循环，所以这种还是通过均分的方式来解决。

首先创建三个掩码，分别为 $m2=01010101$ ， $m4=00110011$ ， $m8=00001111$ 。

对于 x 将其转换为二进制后，思路是：将 x 右移 1 位并和 m2 取与，则统计了每 2 个 bit 中高位 1，清空了低位 1，同样还需要用原来的 $x \& m2$ ，统计低位 1，清空高位 1，然后再把这两个运算的值相加，这样得到的二进制数中，每两个 bit 的二进制数就表示这两个 bit 共有几个 1；

相应的，对于每四个、每八个的运算逻辑也是一致的。所以最后我们会得到 4 组 bit，每组有 8 个 bit。但是接下来的操作就不再需要掩码了。因为 16 的二进制数为 10000，小于八位，不会进位溢出，所以直接让高 4 位和低 4 位相加即可；相应的，最后一步就是高 8 位加低 8 位，那么所有的 1 的总数就被算出来了。

但是到这里还没有结束，总共最多会有 32 个 1，因为像 -1 这种负数，会导致最后算出来的总数有很多垃圾位，所以我们只需要取最终的这个二进制数的低 6 位即可。

P19: bitReverse - Reverse bits in an 32-bit integer

在这道题中，我们要把 x 的二进制位按顺序整体反转。因为不能用循环，所以这种还是通过分块交换的方式来解决。

首先创建四个掩码，分别为 $m1=01010101$ ， $m2=00110011$ ， $m4=00001111$ ， $m8=0000000011111111$ 。

对于 x 将其转换为二进制后，思路是：将 x 右移 1 位并和 m1 取与，就取出了每 2 个 bit 中高位的那个比特，挪到了低位的位置；同样还需要用原来的 $x \& m1$ 之后左移 1 位，取出低位的比特，挪到高位的位置；然后再把这两个运算的结果按位或起来，这样得到的二进制数中，每两个 bit 就完成了左右互换。相应的，对于每四个、每八个的运算逻辑也是一致的。所以最后我们会完成 4bit 组、8bit 组的翻转，再做一次高低 16 位的整体交换，一共五轮操作下来，整个 32 位的比特就全部反转完成了。

和上一题一样，return 部分还需要额外处理，因为 int 是算术右移，像 -1 这种负数，右移的时候高位会补 1，所以每一次右移之后都必须和掩码取与，清掉补出来的符号位；另外最后一步 16 位左移时，原来的高 16 位会直接溢出丢弃，不需要额外再加掩码。

二、测试结果

```
root@ics: ~/DataLab
classifyAdd3: 39/52 operators
floatScaleThreeHalves: 46/60 operators
floatRoundEven: 27/65 operators
float_i2f: 26/40 operators
bitCount: 35/40 operators
bitReverse: 28/34 operators
All 19 functions passed operator checks.
root@ics:~/DataLab# ./btest
Score Rating Errors Function
1 1 0 signMask
2 2 0 bitXor
3 3 0 negativePart
4 4 0 copyByteWithin
4 4 0 logicalShift
4 4 0 swapNibblePairs
4 4 0 secondLowestZeroBit
5 5 0 oddParity
5 5 0 rotateRightBits
5 5 0 roundEvenPow2
5 5 0 midpointTowardFirst
7 7 0 isBetweenEitherOrder
7 7 0 mul5Sat
7 7 0 classifyAdd3
7 7 0 floatScaleThreeHalves
10 10 0 floatRoundEven
10 10 0 float_i2f
10 10 0 bitCount
10 10 0 bitReverse
Total points: 110/110
root@ics:~/DataLab#
```

```
root@ics: ~/DataLab
1 file changed, 238 insertions(+), 21 deletions(-)
root@ics:~/DataLab# make clean
make all
rm -f *.o btest fshow ishow *~
gcc -O -Wall -fwrapv -m32 -o btest bits.c btest.c decl.c tests.c -lm
gcc -O -Wall -fwrapv -m32 -o fshow fshow.c
gcc -O -Wall -fwrapv -m32 -o ishow ishow.c
root@ics:~/DataLab# ./check_ops.py bits.c
signMask: 1/2 operators
bitXor: 7/8 operators
negativePart: 4/6 operators
copyByteWithin: 10/12 operators
logicalShift: 6/20 operators
swapNibblePairs: 18/18 operators
secondLowestZeroBit: 7/8 operators
oddParity: 12/56 operators
rotateRightBits: 16/16 operators
roundEvenPow2: 18/24 operators
midpointTowardFirst: 21/32 operators
isBetweenEitherOrder: 31/48 operators
mul5Sat: 25/30 operators
classifyAdd3: 39/52 operators
floatScaleThreeHalves: 46/60 operators
floatRoundEven: 27/65 operators
float_i2f: 26/40 operators
bitCount: 35/40 operators
bitReverse: 28/34 operators
All 19 functions passed operator checks.
root@ics:~/DataLab# ./btest
Score Rating Errors Function
```